CarePulse Healthcare Management System

Full-Stack Technical Architecture, Relational Schema & Interview Prep Manual

---

Tech Stack: React.js | Python FastAPI | SQLAlchemy ORM | PostgreSQL
Security: JSON Web Token (JWT) | BCrypt Password Hashing | Role-Based Access Control
Verification: Pytest Integration Suite | Postman Collection API Test Client

PREPARED FOR: SDE INTERNSHIP & FRESHER PORTFOLIO DEPLOYMENT
Target Competencies: Full Stack Dev, REST API Design, Database Normalization, Security

Compiled: August 2026

1. Executive Summary & System Architecture

1.1 Project Objective

The CarePulse Healthcare Management System (HMS) is a secure, responsive, full-stack application designed to manage clinical schedules, patient registration, and prescription logs. It demonstrates modern development practices including client-server segregation, relational database design with foreign keys, JWT-based state-isolated authentication, and role-based permissions at both client and server layers. It is built as a production-grade demonstration for an SDE Internship profile.

1.2 Client-Server Architecture

The system adheres to a segregated client-server architecture. The Frontend (React) and Backend (FastAPI) communicate strictly via HTTP REST APIs using the Axios library. The backend acts as a stateless gateway, validating tokens and resolving business rules, while database interactions are abstractly managed through SQLAlchemy ORM, directing queries to a PostgreSQL database server (with SQLite fallback capability for seamless execution in restricted environments).

1.3 Core Modules

1. Admin Module: Enables management of doctor credentials, receptionist registrations, and provides clinic-wide stats.
2. Receptionist Module: Serves as the scheduler, handles patient medical card registrations, and books/updates appointments.
3. Doctor Module: Provides the physician schedule console and enables issuing prescriptions to auto-complete bookings.
4. Patient Module: Provides a portal for patients to view prescription histories and check upcoming appointment timetables.

2. Full Tech Stack Deep Dive

2.1 Frontend Technologies

- React.js (Vite): Provides a fast, hot-reloading development server and compiled single-page application wrapper.
- React Router Dom (v6): Handles nested routing, page switching, and route authorization guards (ProtectedRoute).
- Axios: Communicates with backend endpoints. Configured with a central request interceptor that dynamically binds the JWT token from localStorage to the HTTP Authorization headers.
- Lucide-React: Supplies modular vector icons for consistent modern UI styling.
- Vanilla CSS3: Declares responsive layouts, glassmorphism cards, scrollbar formatting, and micro-animations through CSS custom properties.

2.2 Backend Technologies

- FastAPI (Python 3.10): Leverages asynchronous execution, yields high performance, and auto-generates OpenAPI schemas.
- Uvicorn: Serves as the ASGI web server managing HTTP connections.
- SQLAlchemy (v2.0): Provides database engine abstractions, object-relational mapping, and session transaction controllers.
- PyJWT (python-jose): Encodes, decodes, and verifies cryptographic JWT tokens.
- Passlib (bcrypt): Manages password security using salt-strengthened hashing algorithms.

2.3 Database Management

- PostgreSQL: Serves as the relational database engine, enforcing constraints, primary keys, and foreign keys.
- SQLite: Configured as an automatic local fallback database. If PostgreSQL port connection throws an error on startup, the SQLAlchemy connection engine dynamically shifts to a local SQLite database, ensuring high portability.

3. Database Relational Schemas

3.1 Users Table

Stores accounts for authentication.
- id: Integer, Primary Key, Auto Increment
- username: String, Unique, Indexed (serves as login identifier)
- password_hash: String (cryptographically hashed)
- role: String ('Admin' | 'Receptionist' | 'Doctor' | 'Patient')
- created_at: DateTime, defaults to now()

3.2 Doctors Table

Stores physician metadata.
- id: Integer, Primary Key, Auto Increment
- user_id: Integer, Foreign Key -> users.id (Unique, nullable, cascade delete)
- full_name: String, email: String (Unique)
- department: String (Cardiology, Pediatrics, etc.)
- experience: Integer (years)
- available_slots: JSON array of strings (e.g. ['09:00', '10:00'])

3.3 Patients Table

Stores patient records.
- id: Integer, Primary Key, Auto Increment
- user_id: Integer, Foreign Key -> users.id (Unique, nullable, cascade delete)
- full_name: String, age: Integer, gender: String
- phone_number: String, email: String (Unique)

3.4 Appointments Table

Schedules and tracks consultations.
- id: Integer, Primary Key, Auto Increment
- patient_id: Integer, Foreign Key -> patients.id (cascade delete)
- doctor_id: Integer, Foreign Key -> doctors.id (cascade delete)
- date: Date, time: String (shift slot)
- status: String ('Booked' | 'Completed' | 'Cancelled')

3.5 Prescriptions Table

Contains prescription details.
- id: Integer, Primary Key, Auto Increment
- patient_id: Integer, Foreign Key -> patients.id
- doctor_id: Integer, Foreign Key -> doctors.id
- appointment_id: Integer, Foreign Key -> appointments.id (nullable)
- medicine_name: String, dosage: String, duration: String, doctor_notes: Text
- created_at: DateTime, defaults to now()

4. API Documentation & Endpoints Catalog

4.1 Authentication Endpoints

- POST /register : Public endpoint. Creates a user login and patient profile card.
- POST /login : Public. Validates credentials, issues JWT token containing role, user_id, and username.
- POST /logout : Validates token, logs out session.
- GET /me : Returns current logged-in user profile metadata and mapping keys.

4.2 Patients Registry (Scoped Access)

- GET /patients : Admin, Receptionist, Doctor. Lists and searches patients registry.
- GET /patients/{id} : Scoped. Allowed for staff or corresponding Patient (self).
- POST /patients : Receptionist, Admin. Adds patient profile and creates User account.
- PUT /patients/{id} : Scoped. Updates details (also updates password if supplied).
- DELETE /patients/{id} : Admin, Receptionist. Deletes patient profile and credentials.

4.3 Doctors Catalog

- GET /doctors : Public (to select doctors). Lists doctors and departments.
- GET /doctors/{id} : Authenticated. Returns single doctor profile.
- POST /doctors : Admin. Creates doctor profile and corresponding User login.
- PUT /doctors/{id} : Admin. Updates doctor details and shifts.
- DELETE /doctors/{id} : Admin. Removes doctor from directory.

4.4 Appointments & Prescriptions

- GET /appointments : Scoped to role. Admin/Receptionist sees all. Doctor/Patient see their own list.
- POST /appointments : Receptionist, Admin. Book consultation. Triggers double-booking validation.
- PUT /appointments/{id} : Staff & Doctor. Reschedule or change status (Complete/Cancel).
- DELETE /appointments/{id} : Cancel appointment (sets status to 'Cancelled').
- GET /prescriptions/{patientId} : Doctors & Patient (self). View prescriptions list.
- POST /prescriptions : Doctor. Issue prescription. Auto-completes the corresponding appointment.

5. Security, RBAC & Core Logic

5.1 JWT-Based Authentication

Token generation occurs on successful login. The token payload stores user identity ('sub': username) and role ('role'). The backend cryptographically signs this token using HMAC SHA256 and a server-side secret key. FastAPI checks this signature on subsequent requests, extracting the current active user from the db. The frontend Axios client intercepts responses, catching 401 errors, clearing credentials, and redirecting to the login screen.

5.2 Role-Based Access Control (RBAC)

Guards are applied at two levels:
1. Database API layer: FastAPI dependencies (require_admin, require_receptionist, require_doctor, require_patient) validate roles. Unauthorized attempts trigger HTTP 403 Forbidden errors.
2. UI layer: ProtectedRoute React wrapper checks user role from local storage. Non-authorized route transitions redirect to the dashboard.

5.3 Double-Booking Prevention Logic

To avoid scheduling overlaps, booking checks are executed atomically in the appointments router. Before completing a POST or PUT booking operation, the system queries the database for existing appointments with identical doctor_id, date, time, and status == 'Booked'. If an overlap is detected, an HTTP 400 Bad Request is returned to prevent conflicting schedules.

5.4 Prescription-Appointment Integration

When a physician submits a prescription, they link it to the current appointment ID. The backend router saves the prescription and automatically updates the related appointment's status to 'Completed'. This ensures the schedule updates in real-time without requiring manual state changes by the receptionist.

6. SDE Portfolio Interview Q&A Cheatsheet

Q1: How did you implement double-booking validation?

Answer: Double-booking validation is handled atomically on the backend in the appointments router. Before saving a booking request, we query the DB to check if the doctor has any active appointments (status == 'Booked') on the same date and time. If a match is found, we raise an HTTP 400 Bad Request with a message: 'Doctor is already booked for this date and time slot.' For updates, we run the same query but exclude the current appointment ID (Appointment.id != current_id) to allow rescheduling to different slots without self-conflict.

Q2: Why did you use JWT instead of Session Cookies?

Answer: JWT (JSON Web Token) is stateless and scales better in modern architectures. By signing user metadata (username, role) inside the token, the backend does not need to store session states in a database or cache (like Redis). The client stores the token in localStorage and attaches it to request headers. The backend decodes and validates the signature using the SHA256 secret key, verifying authentication statelessly. If the project scales to multiple server instances, we don't have to worry about session replication.

Q3: How does your database fallback mechanism function?

Answer: To ensure high portability and ease of evaluation for recruiters, the backend database engine in database.py connects dynamically. It tries to establish a connection with the PostgreSQL server using the URL from the environment variables (with a 3-second timeout). If the connection fails (e.g. PostgreSQL is not installed or running locally), it catches the exception, logs a warning, and falls back to a local SQLite database file (sqlite:///./healthcare.db). The system automatically runs SQLAlchemy schema migrations on the active database engine to create tables, followed by database seeding.

Q4: How does issuing a prescription affect appointment statuses?

Answer: I integrated the prescription and appointment flows. When a doctor writes a prescription post-consultation, they provide the patient_id, doctor_id, and appointment_id. When the API handler receives this request, it inserts the prescription record and updates the status of the corresponding appointment to 'Completed' in the same database transaction. This automates the clinical workflow and updates receptionist dashboards immediately.

7. Limitations, Scalability & Future Work

7.1 System Limitations

1. Stateless JWT Revocation: Because JWTs are stateless, once issued, they remain valid until they expire. If a user logs out, the frontend deletes the token, but if the token was intercepted, it could theoretically still access endpoints. Implementing a Redis-based blacklist database would solve this.
2. Local File System SQLite Fallback: While convenient for portfolio runs, SQLite doesn't support concurrent writes at scale, causing database locking issues in high-traffic applications. Production must enforce PostgreSQL.
3. Lack of Real-Time Updates: The frontend dashboard counts and lists rely on page reloads or API polling. It lacks push notifications.

7.2 Scalability Challenges

1. Database Read-Write Bottlenecks: In a real clinic, doctor/patient directory reads are high, while appointment writes are frequent. To scale, we must implement database replication (Primary writes, Replica reads) with SQLAlchemy routing.
2. Scheduling Overlaps under High Concurrency: If two receptionists click 'Confirm' on the same slot at the exact same millisecond, a race condition could bypass the Python conflict check. We would need to enforce database transaction isolation levels (Serializable) or use Redis distributed locks (Redlock) for time slots.
3. Monolith Splits: To support millions of queries, the modules (Auth, Doctor scheduling, Patients EHR records, Billing) should be split into microservices, communicating via message queues like RabbitMQ or Kafka.

7.3 Future Work & Enhancements

1. HL7 FHIR Protocol Integration: Align patient EHR cards with HL7 FHIR (Fast Healthcare Interoperability Resources) medical standards to support data exchanges with hospitals.
2. WebSockets Real-Time Queues: Implement WebSocket connections so receptionists see patient check-ins and doctors see live queue updates immediately.
3. Video Consultation: Integrate WebRTC (e.g., Twilio or Zoom API) directly into the Doctor and Patient portals for remote telehealth appointments.
4. Push Reminders: Connect SendGrid and Twilio SMS APIs to send automated email/SMS reminders to patients 24 hours before their appointments, reducing clinic no-shows.